

```
# Cell: Mount Google Drive
# Import the drive module from google.colab
from google.colab import drive
# Mount Google Drive to the /content/drive directory
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
# Cell: List GEE EXPORTS Folder Contents
# Use a shell command to list the contents of the specified Google Drive folder
!ls "/content/drive/MyDrive/GEE_EXPORTS (ROAD CONSTRUCTION)"
```

```
detected_road_construction.tif  'Road_Progress_S2_2026 (1).tif'
road_corridor_mask4.tif        Road_Progress_S2_2026_AFTER.tif
road_corridor_mask.tif         Road_Progress_S2_2026_BEFORE.tif
road_corridor.shp              Road_Progress_S2_2026.tif
```

```
# Cell: Import Core Libraries
# Import the rasterio library for reading and writing raster datasets
import rasterio
# Import the numpy library for numerical operations
import numpy as np
# Import the matplotlib.pyplot library for plotting and visualization
import matplotlib.pyplot as plt
```

```
# Cell: Define Image and Corridor File Paths
# Define the file path for the 'before' image
before_img = "/content/drive/MyDrive/GEE_EXPORTS (ROAD CONSTRUCTION)/Road_Progress_S2_2026_BEFORE.tif"
# Define the file path for the 'after' image
after_img = "/content/drive/MyDrive/GEE_EXPORTS (ROAD CONSTRUCTION)/Road_Progress_S2_2026_AFTER.tif"
# Define the file path for the road corridor mask
corridor = "/content/drive/MyDrive/GEE_EXPORTS (ROAD CONSTRUCTION)/road_corridor_mask4.tif"
```

```
# Cell: Define NDVI Computation Function
# Import numpy for numerical operations
import numpy as np
# Import rasterio for working with raster data
import rasterio

# Define a function to compute NDVI from a given image path
def compute_ndvi(path):
    # Open the raster image using rasterio
    with rasterio.open(path) as src:
        # Read the Red band (Band 3 for Sentinel-2) and convert to float32
        red = src.read(3).astype("float32") # Band 4
        # Read the Near-Infrared band (Band 4 for Sentinel-2) and convert to float32
        nir = src.read(4).astype("float32") # Band 8
        # Get the nodata value from the source image
        nodata = src.nodata
        # Get the profile (metadata) of the source image
```

```
profile = src.profile

# Create a mask for valid pixels (where both red and nir are finite numbers)
mask = np.isfinite(red) & np.isfinite(nir)

# Initialize ndvi array with NaNs, same shape as red/nir bands
ndvi = np.full(red.shape, np.nan, dtype="float32")
# Compute NDVI only for valid pixels to avoid division by zero or NaNs
ndvi[mask] = (nir[mask] - red[mask]) / (nir[mask] + red[mask] + 1e-6)

# Return the computed NDVI array and the image profile
return ndvi, profile
```

```
# Cell: Compute NDVI for Before and After Images
# Compute NDVI for the 'before' image and store the result and profile
ndvi_before, profile = compute_ndvi(before_img)
# Compute NDVI for the 'after' image and store the result (profile is ignored with _)
ndvi_after, _ = compute_ndvi(after_img)
```

```
# Cell: Calculate NDVI Change
# Compute the difference between 'before' and 'after' NDVI values
ndvi_change = ndvi_before - ndvi_after
```

```
# Cell: Inspect NDVI Change Range
# Find and print the minimum (nanmin) and maximum (nanmax) non-NaN values in the ndvi_change array
np.nanmin(ndvi_change), np.nanmax(ndvi_change)

(np.float32(-0.13886786), np.float32(0.29920062))
```

```
# Cell: Identify Potential Construction Areas
# Create a boolean mask where ndvi_change is greater than a threshold (0.1)
# This indicates areas where vegetation likely decreased, suggesting construction
construction = ndvi_change > 0.1
```

```
# Cell: Combine Construction and Corridor Masks
# Create the final mask by performing a logical AND operation between 'construction' and 'corridor_mask'
# This ensures only areas within the road corridor showing construction signs are included
final_mask = construction & corridor_mask
```

```
# Cell: Re-inspect NDVI Change Range (Duplicate)
# Find and print the minimum (nanmin) and maximum (nanmax) non-NaN values in the ndvi_change array
# This cell is a duplicate of s4KH2-r9Y_Ka for inspecting the range again
np.nanmin(ndvi_change), np.nanmax(ndvi_change)
```

```
# Cell: Check Unique Values in Final Mask
# Print the unique values present in the 'final_mask' (expected to be True/False)
np.unique(final_mask)
```

```
# Cell: Visualize Final Mask with Matplotlib
# Create a new figure with a specified size
plt.figure(figsize=(6,6))
# Display the 'final_mask' using matplotlib's imshow, with a grayscale colormap
plt.imshow(final_mask, cmap="gray")
# Set the title of the plot
plt.title("Road Construction Detected (NDVI + Corridor)")
# Turn off the axis labels and ticks
plt.axis("off")
# Show the plot
plt.show()
```

```
# Cell: Calculate Detected Construction Area
# Define the area of a single pixel in square meters (10m x 10m = 100 m²)
pixel_area = 10 * 10 # m²
# Calculate the total area in square meters by summing 'True' pixels in 'final_mask' and multiplying by pixel_area
area_m2 = final_mask.sum() * pixel_area
# Print the calculated area in square meters
area_m2
```

```
np.int64(259400)
```

```
# Cell: Estimate Road Length
# Estimate the road length by dividing the total area by an assumed road width (e.g., 20 meters)
road_length_m = area_m2 / 20
# Print the estimated road length in meters
road_length_m
```

```
np.float64(12970.0)
```

```
# Cell: Verify Contents of Generated Mask GeoTIFF
# Import rasterio for reading raster data
import rasterio
# Import numpy for numerical operations
import numpy as np

# Open the generated output mask GeoTIFF file
with rasterio.open(output_mask_path) as src_mask:
    # Read the first band of the mask into a numpy array
    verified_mask_data = src_mask.read(1)
    # Print the unique values present in the mask data
    print(f"Unique values in the generated GeoTIFF: {np.unique(verified_mask_data)}")
    # Print the data type of the mask data
    print(f"Data type of the generated GeoTIFF: {verified_mask_data.dtype}")
    # Print the sum of 'True' (1) pixels, which should correspond to detected construction
    print(f"Sum of detected construction pixels (value 1): {verified_mask_data.sum()}")
```

```
Unique values in the generated GeoTIFF: [0 1]
Data type of the generated GeoTIFF: int8
Sum of detected construction pixels (value 1): 2594
```

```
# Cell: Calculate Total Corridor Area
# Calculate the total area of the corridor in square meters
total_corridor_area_m2 = corridor_mask.sum() * pixel_area

# Print the total corridor area
print(f"Total corridor area: {total_corridor_area_m2:.2f} m²")
```

Total corridor area: 979900.00 m²

```
# Cell: Calculate Percentage of Disturbance
# Calculate the percentage of the disturbed area relative to the total corridor area
percentage_disturbance = (area_m2 / total_corridor_area_m2) * 100

# Print the percentage of disturbance
print(f"Percentage of disturbance: {percentage_disturbance:.2f}%")
```

Percentage of disturbance: 26.47%

```
# Cell: Load Road Corridor Mask from GeoTIFF
# Open the corridor GeoTIFF file using rasterio
with rasterio.open(corridor) as src_corridor:
    # Read the first band of the corridor mask and convert it to a boolean NumPy array
    corridor_mask = src_corridor.read(1).astype(bool)
```

```
# Cell: Save Detected Construction Mask as GeoTIFF
# Import rasterio for handling raster data
import rasterio
# Import folium (though not directly used in this cell, it's for subsequent map visualization)
import folium
# Import Window from rasterio (not directly used here, but common in raster operations)
from rasterio.windows import Window

# Define the output path for the detected construction mask GeoTIFF
output_mask_path = "/content/drive/MyDrive/GEE_EXPORTS (ROAD CONSTRUCTION)/detected_road_construction.tif"

# Open one of the original images (before_img) to get its profile (metadata) as a base
with rasterio.open(before_img) as src:
    base_profile = src.profile.copy()

# Update the base profile for the new GeoTIFF mask
base_profile.update(
    dtype=rasterio.int8, # Set data type to int8 for a boolean mask (0 or 1 values)
    count=1,             # Set band count to 1 (single-band mask)
    nodata=0             # Set the nodata value to 0 for areas with no construction
)

# Write the final_mask to a new GeoTIFF file using the updated profile
with rasterio.open(output_mask_path, 'w', **base_profile) as dst:
    # Write the final_mask (converted to int8) to the first band of the new GeoTIFF
    dst.write(final_mask.astype(rasterio.int8), 1)
```

```
# Print a confirmation message with the output file path
print(f"Detected road construction mask saved to: {output_mask_path}")
```

```
Detected road construction mask saved to: /content/drive/MyDrive/GEE_EXPORTS (ROAD CONSTRUCTION)/detected_road_construction.tif
```

Now, let's visualize this GeoTIFF on an interactive map using `folium`.

```
# -----
# FULL WORKING FOLIUM OVERLAY FIX
# -----

import folium
import rasterio
from rasterio.warp import calculate_default_transform, reproject, Resampling
from rasterio.crs import CRS
import numpy as np
from PIL import Image
import io
import base64

# Open original raster
with rasterio.open(output_mask_path) as src:
    mask_data = src.read(1)
    src_crs = src.crs
    src_transform = src.transform
    src_bounds = src.bounds

    # Define destination CRS (WGS84)
    dst_crs = CRS.from_epsg(4326)

    # Calculate transform for reprojection
    transform, width, height = calculate_default_transform(
        src_crs, dst_crs, src.width, src.height, *src_bounds
    )

    # Prepare empty array for reprojected data
    reprojected = np.empty((height, width), dtype=mask_data.dtype)

    # Reproject raster
    reproject(
        source=mask_data,
        destination=reprojected,
        src_transform=src_transform,
        src_crs=src_crs,
        dst_transform=transform,
        dst_crs=dst_crs,
        resampling=Resampling.nearest
    )

# Create RGBA image
height, width = reprojected.shape
rgba_image = np.zeros((height, width, 4), dtype=np.uint8)
```

```
rgba_image[reprojected == 0] = [0, 0, 0, 0]      # Transparent
rgba_image[reprojected == 1] = [0, 255, 0, 255]    # Lime Green

# Convert to PNG base64
img_pil = Image.fromarray(rgba_image, 'RGBA')
img_byte_arr = io.BytesIO()
img_pil.save(img_byte_arr, format='PNG')
encoded_image = base64.b64encode(img_byte_arr.getvalue()).decode('ascii')
image_data_uri = 'data:image/png;base64,' + encoded_image

# Get new bounds in EPSG:4326
left, bottom = transform * (0, height)
right, top = transform * (width, 0)

folium_bounds = [[bottom, left], [top, right]]

# Center map
center_lat = (bottom + top) / 2
center_lon = (left + right) / 2

# Create map
m = folium.Map(location=[center_lat, center_lon], zoom_start=14)

# Add satellite layer
folium.TileLayer(
    tiles='https://mt1.google.com/vt/lyrs=s&x={x}&y={y}&z={z}',
    attr='Google Satellite',
    name='Google Satellite',
    overlay=False,
    control=True
).add_to(m)

# Add overlay
folium.raster_layers.ImageOverlay(
    image=image_data_uri,
    bounds=folium_bounds,
    opacity=0.9,
    name='Detected Road Construction'
).add_to(m)

folium.LayerControl().add_to(m)

m
```



```
/tmp/ipykernel_365/313150945.py:51: DeprecationWarning: 'mode' parameter is deprecated and will be removed in Pillow 13 (2026-10-15)  
img_pil = Image.fromarray(rgba_image, 'RGBA')
```

